

Vision-Based Precision Pose Estimation for Autonomous Formation Flying

UE24CS352A – Machine Learning Mini-Project

Shashank D — PES1UG24CS433

Shriya K — PES1UG24CS448

1. Problem Statement

The project addresses relative pose estimation for autonomous formation flying. A follower aircraft uses observations of a leader aircraft from a monocular camera to estimate relative position and orientation. The system combines a learned discrete pose classifier with model-based filtering and geometric refinement.

2. Dataset and Feature Representation

A synthetic dataset is generated from a 3D aircraft model and a simulated monocular gimbaled camera. Fourteen predefined structural keypoints are projected into the camera image. Each keypoint contributes its image coordinates (u, v) and an occlusion indicator, producing a 42-dimensional input vector. The pose space is discretized into 10 X bins, 10 Y bins, 8 Z bins, and 6 roll bins, giving 4,800 discrete pose classes.

The camera simulation includes perspective projection, image-boundary checks, geometric occlusion handling, and Gaussian pixel noise. The current implementation begins with structural feature measurements rather than a raw-image CNN feature detector.

3. Methodology

The baseline ANN maps the 42-dimensional feature vector through two hidden layers of 100 neurons to a 4,800-way softmax output. ReLU activations, cross-entropy loss, and Adam optimization are used. The predicted class provides a coarse pose estimate.

A particle filter is then used for temporal/model-based refinement. Particles are initialized around the ANN estimate, propagated using the simulated motion model, evaluated by feature reprojection error, weighted by likelihood, and resampled.

The proposed extension applies Perspective-n-Point (PnP) refinement using known 3D aircraft keypoints and detected 2D image correspondences. The coarse pose is used to initialize an iterative solvePnP optimization, producing a continuous pose estimate. The simulated camera is modeled as gimbaled, with its optical axis tracking the leader line of sight.

4. Implementation

The implementation is organized into simulation, geometry, filtering, evaluation, training, and test modules. Python, NumPy, PyTorch, and OpenCV are used. The evaluation framework executes a 10-second simulated formation-flight trajectory with 100 steps at 10 Hz and compares the ANN, ANN + particle filter, and ANN + PnP methods using position and roll errors together with execution latency.

5. Quantitative Results

Method	Pos RMSE (m)	Pos Median (m)	Pos Max (m)	Roll RMSE (°)	Latency (ms)	FPS
Discrete ANN	22.549	17.402	40.342	10.75	0.50	1986.4
ANN + Particle Filter	24.469	23.865	36.943	3.26	220.41	4.5
ANN + PnP Refiner	0.644	0.380	1.888	0.54	0.41	2416.9

6. Discussion

The benchmark shows that the ANN provides a fast coarse estimate but retains substantial positional error because the output is discretized. The particle filter reduces roll error relative to the ANN, but its computational cost is much higher and its positional RMSE is higher in this trajectory. The PnP refinement produces the strongest continuous-pose result in the current synthetic benchmark, combining geometric correspondence with the coarse learned estimate.

The camera model assumes a gimbaled sensor whose optical axis follows the leader line of sight. This is a simulation assumption and should be treated as known camera/gimbal state when interpreting the PnP results. The benchmark therefore evaluates the proposed refinement under the stated gimbaled-camera model rather than claiming an end-to-end raw-image pose estimator.

7. Conclusion

The project demonstrates a hybrid learning-and-geometry pipeline for relative aircraft pose estimation. ANN classification provides rapid coarse localization, particle filtering supplies temporal/model-based refinement, and PnP provides continuous geometric refinement. In the current simulated experiment, the ANN + PnP method achieves the lowest reported position and roll errors while retaining very low measured inference latency.

Future work includes direct keypoint detection from raw camera images, evaluation on real or hardware-in-the-loop flight data, robust handling of mismatched/outlier keypoints, uncertainty estimation, and bundle-adjustment-based refinement.

Reproducibility

Run the trained benchmark with: `python -m src.evaluation.benchmark` The benchmark generates the flight-trajectory and comparative-error figures under `docs/figures/`. The repository also contains unit tests for the implemented components.